1. Introduction

In recent years, deep learning has achieved great success in fields such as image recognition and natural language processing and is widely used as a powerful learning method for expressing complex non-linearity. However, its learning process presents many challenges, such as setting initial values for weights and biases, selecting optimization algorithms, and designing network structures. In particular, learning based on the backpropagation method may face problems such as vanishing gradients and convergence to local optimal solutions.

Therefore, this research focuses on the Probabilistic Neural Network (PNN), which has the potential to overcome these challenges. PNN is capable of high-speed learning with only a single smoothing parameter and performs classification based on probabilistic output, making it robust to the effects of outliers. However, in conventional PNN, the training data is stored as is in the pattern layer, which poses a problem as the classification processing time increases with the number of data points. This study aims to accelerate this processing time by utilizing parallel processing on CPUs and GPUs.

2. Principles and Methods

2.1 Difference between CPU and GPU

A major difference between a CPU and a GPU lies in their parallelism. A CPU has one or a few cores (processing units) and executes instructions one by one. In contrast, a GPU has hundreds to thousands of cores and can execute many instructions simultaneously.

2.2 CUDA

CUDA is a parallel computing platform developed by NVIDIA in 2007 to provide an environment where NVIDIA GPUs can perform optimally. It enables the acceleration of parallel processing on GPUs and is used in various fields such as scientific and technical computing, machine learning, big data analysis, and image/video processing.

2.3 PNN

PNN, proposed by Specht, is a type of multilayer neural network composed of an input layer, a pattern layer, a summation layer, and an output layer. Figure 1 shows the structure of a PNN.

2.4 Varying the Number of Threads and Measuring Processing Time

The number of threads represents the maximum number of instructions that can be processed simultaneously. In the case of an 8-core/16-thread CPU, while 8 instructions can be processed at once, each core can handle up to two, allowing a maximum of 16 instructions to be processed simultaneously. For this reason, the number of cores is also referred to as the number of physical cores, and the number of threads as the number of logical cores.

In this experiment, a parallel experiment on the CPU was conducted using Python. The number of threads in the PNN program was varied, and the processing time was measured and compared. A parallel experiment was also conducted on the GPU, and the results were compared.

In the GPU parallel processing, the x-y dimensions of the block, which is a collection of threads, were varied, and the processing time was measured and compared. Since the maximum value for a block in the GPU used for this experiment

was 1024, the x and y values were set such that their product was 1024.

3. Results

3.1 Execution Environment

The experiment used a program written in the Python language for pattern classification with PNN in a parallel environment on the CPU, and a program using the C++ language and CUDA for the parallel environment on the GPU. Table 1 shows the specifications of the PC used for the experiment and the execution environment.

3.2 Experimental Results

As described in section 2.4, the number of threads was varied, and the classification processing time for each dataset was measured. Next, a parallel experiment was also conducted on the GPU in the same manner as the CPU case.

The results of varying the GPU's x-y dimensions during measurement are shown in Table 3.

4. Conclusion

The results of the CPU parallel experiment show that increasing the number of threads from 1 generally speeds up the processing. However, for some datasets, increasing the number of threads too much resulted in slower processing times. This phenomenon was particularly noticeable for datasets with a smaller computational load. Therefore, while data with a heavy computational load can be maximally parallelized, it is conceivable that when the number of threads is too high, the impact of overhead from parallelization becomes a factor.

Next, regarding the comparison between GPU and CPU parallel processing, it was predicted that the CPU would be faster for datasets with small amounts of data and the GPU would be faster for those with large amounts. However, the experiment showed that the GPU was faster in most cases.

Furthermore, the results from varying block dimensions show that an x-y dimension of 32-32 was the fastest. The reason for this is that the MNIST image size is 28x28, and a 32x32 block allows the image data to be completely loaded into shared memory. This makes it possible to perform the processing entirely within the fast shared memory, completely avoiding access to global memory.

From the above, it is clear that the problem of long processing times for PNN pattern classification can be addressed by CPU parallelization for each dataset in this experiment. However, because the GPU processing took longer than expected, it was not possible to distinguish which datasets were better suited for the CPU and which for the GPU. Possible reasons for this include the fact that the programming languages used for the parallel processing programs on the CPU and GPU were different, which may have led to differences in internal processing. Another factor could be that during the C++ implementation, where both CPU and GPU processing times were measured, the CPU was already faster in many cases.

Therefore, future tasks include implementing and comparing the programs in the same language, reviewing the programs, and verifying whether the differences were due to the PC environment used.

使用AI　gemini pro2.5
一段落ごとに英語に直してもらった
図省略

評価方法
英文が論文に適しているか判断するのは難しいので、英語にしてもらったものを、Google翻訳を使って同じ意味になるかを調べた。
例えば、最初の部分は

1. はじめに
近年、深層学習は画像認識や自然言語処理などの分野で大きな成果を上げており、複雑な非線形性を表現する強力な学習手法として広く利用されています。しかし、その学習プロセスには、重みやバイアスの初期値設定、最適化アルゴリズムの選択、ネットワーク構造の設計など、多くの課題があります。特に、バックプロパゲーション法に基づく学習では、勾配消失や局所最適解への収束といった問題に直面する可能性があります。
そこで本研究では、これらの課題を克服する可能性を持つ確率ニューラルネットワーク（PNN）に焦点を当てます。PNNは、単一の平滑化パラメータのみで高速学習が可能であり、確率的な出力に基づいて分類を行うため、外れ値の影響に対して頑健です。しかし、従来のPNNでは、学習データがパターン層にそのまま格納されるため、データ点数の増加に伴い分類処理時間が増大するという問題があります。本研究では、CPUやGPUによる並列処理を活用することで、この処理時間の高速化を目指します。

となり、若干言葉は違うが、ほとんど同じ意味になった。その後の文章も同じような結果となった。

また、英文が論文にふさわしいかを判断するためにGeminiにもう一度通したり、claudeを使用したが、一文が正しいという根拠を持ってこれなかったため、判断は不可。